

Evidencias de ejecución en Terminal

```
Arbol de nodos
1.- Insertar expresión posfija
2.- Mostrar preorden
3.- Mostrar inorden
4.- Mostrar posorden
5.- Salir
Selecciona una opcion: 1
Ingresa la expresión posfija: 3 4 +
Se ha insertado la expresión en el árbol.
Resultado de la operacion: 7
```

```
Recorrido en inorden:
(3 + 4)
```

```
Arbol de nodos
1.- Insertar expresión posfija
2.- Mostrar preorden
3.- Mostrar inorden
4.- Mostrar posorden
5.- Salir
Selecciona una opcion: 1
Ingresa la expresión posfija: 8 2 / 3 4 + *
Se ha insertado la expresión en el árbol.
Resultado de la operacion: 28.0
```

```
Recorrido en inorden:
((8 / 2) * (3 + 4))
```

```
Arbol de nodos
1.- Insertar expresión posfija
2.- Mostrar preorden
3.- Mostrar inorden
4.- Mostrar posorden
5.- Salir
Selecciona una opcion: 1
Ingresa la expresión posfija: 3 4 + 5 2 - *
Se ha insertado la expresión en el árbol.
Resultado de la operacion: 21
```

```
Recorrido en inorden:
((3 + 4) * (5 - 2))
```

Arbol de nodos

- 1.- Insertar expresión posfija
- 2.- Mostrar preorden
- 3.- Mostrar inorden
- 4.- Mostrar posorden
- 5.- Salir

Selecciona una opcion: 1

Ingresa la expresión posfija: 10 2 3 * +

Se ha insertado la expresión en el árbol.

Resultado de la operacion: 16

Recorrido en inorden:
(10 + (2 * 3))

Arbol de nodos

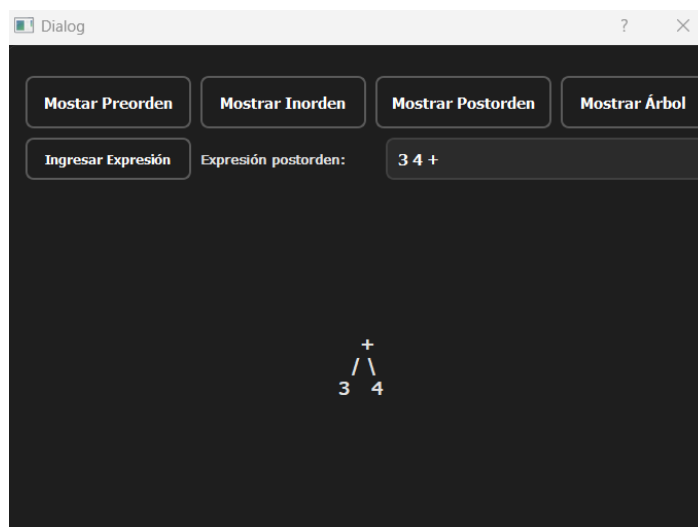
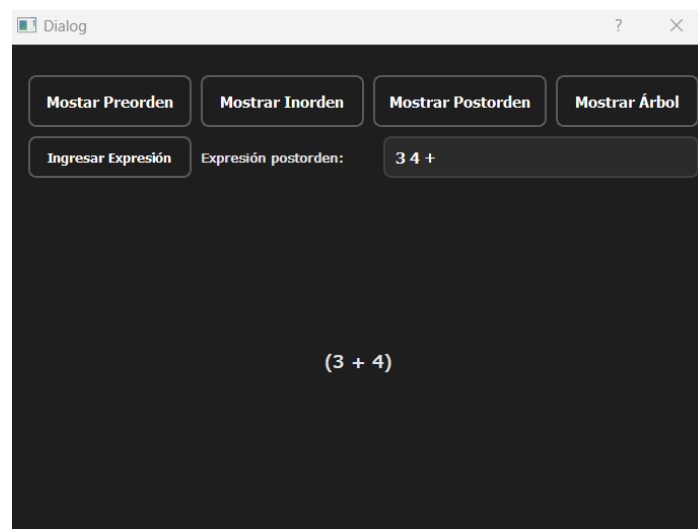
- 1.- Insertar expresión posfija
- 2.- Mostrar preorden
- 3.- Mostrar inorden
- 4.- Mostrar posorden
- 5.- Salir

Selecciona una opcion: 1

Ingresa la expresión posfija: 3 +

Expresión inválida.

Evidencias de ejecución de Interfaz



Dialog ? X

Mostrar Preorden

Mostrar Inorden

Mostrar Postorden

Mostrar Árbol

Ingresar Expresión

Expresión postorden: 8 2 / 3 4 + *

Se ha insertado la expresión en el árbol.
Resultado de la operación: 28.0

Dialog ? X

Mostrar Preorden

Mostrar Inorden

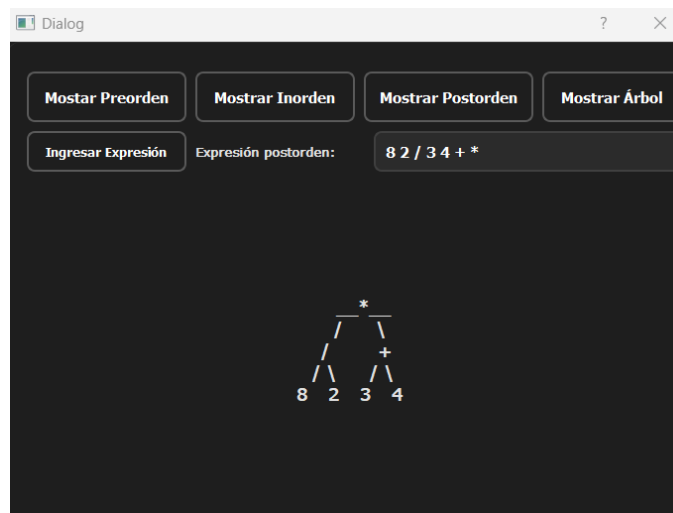
Mostrar Postorden

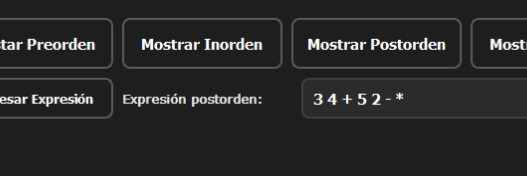
Mostrar Árbol

Ingresar Expresión

Expresión postorden: 8 2 / 3 4 + *

$((8 / 2) * (3 + 4))$





Dialog

?

X

Mostrar Preorden

Mostrar Inorden

Mostrar Postorden

Mostrar Árbol

Ingresar Expresión

Expresión postorden:

3 4 + 5 2 - *

Se ha insertado la expresión en el árbol.
Resultado de la operacion: 21

Dialog

Mostrar Preorden

Mostrar Inorden

Mostrar Postorden

Mostrar Árbol

Expresión postorden:

3 4 + 5 2 - *

$((3 + 4) * (5 - 2))$

The screenshot shows the "Dialog" application window. It has a dark theme. At the top, there are four buttons: "Mostrar Preorden", "Mostrar Inorden", "Mostrar Postorden", and "Mostrar Árbol". Below these is a label "Expresión postorden:" followed by a text input field containing the expression "3 4 + 5 2 - *". At the bottom center, a binary tree diagram is displayed, representing the postfix expression. The root node is labeled "*", with left child "+" and right child "-". The "+" node has children "3" and "4", and the "-" node has children "5" and "2".

Dialog

?

×

Mostrar Preorden

Mostrar Inorden

Mostrar Postorden

Mostrar Árbol

Ingresar Expresión

Expresión postorden:

10 2 3 * +

Se ha insertado la expresión en el árbol.
Resultado de la operación: 16

Dialog

?

×

Mostrar Preorden

Mostrar Inorden

Mostrar Postorden

Mostrar Árbol

Ingresar Expresión

Expresión postorden:

10 2 3 * +

$(10 + (2 * 3))$

Dialog

?

×

Mostrar Preorden

Mostrar Inorden

Mostrar Postorden

Mostrar Árbol

Ingresar Expresión

Expresión postorden:

10 2 3 * +

```
graph TD; A["+"] --- B["10"]; A --- C["*"]; C --- D["2"]; C --- E["3"];
```

Dialog

?

×

Mostrar Preorden

Mostrar Inorden

Mostrar Postorden

Mostrar Árbol

Ingresa Expresión

Expresión postorden:

3 +

Expresión inválida.

Dialog

?

×

Mostrar Preorden

Mostrar Inorden

Mostrar Postorden

Mostrar Árbol

Ingresa Expresión

Expresión postorden:

3 +

El árbol está vacío.

Prompt utilizado

Estoy implementando en Python un árbol binario de expresión a partir de una expresión en notación postfija. El algoritmo que comprendí es el siguiente:

1. Recorrer cada token.
2. Si es un operando, crear un nodo y colocarlo en una pila.
3. Si es un operador, extraer primero el hijo derecho y después el hijo izquierdo.
4. Crear un nodo operador, conectar sus hijos y colocarlo nuevamente en la pila.
5. Al terminar, el único nodo de la pila será la raíz. Revisa únicamente si la lógica es correcta. No generes todavía el código. Indica posibles errores o casos que debo considerar.

Respuesta obtenida

Sí, la lógica que describiste es la forma estándar de construir un árbol de expresión a partir de una expresión en notación postfija. El algoritmo es correcto.

Hay algunos detalles y casos que conviene considerar antes de implementarlo:

1. **El orden de los hijos es muy importante.**
 - Cuando encuentres un operador, debes extraer primero el operando que será el **hijo derecho**.
 - Después extraes el que será el **hijo izquierdo**.
 - Si inviertes este orden, operaciones no conmutativas como la resta (-) o la división (/) producirán resultados incorrectos.

Por ejemplo:

8 3 -

La pila contiene:

8

3

Al encontrar -:

- derecho = 3
- izquierdo = 8

Se forma:

-

/\

8 3

2. **Debe existir un número suficiente de operandos.**

Antes de sacar dos nodos de la pila, debe haber al menos dos elementos.

Ejemplo inválido:

5 +

Aquí solo existe un operando y el árbol no puede construirse.

3. **Al finalizar debe quedar exactamente un nodo.**

Si quedan dos o más nodos en la pila, la expresión postfija es inválida.

Ejemplo:

A B C

Nunca apareció un operador para unirlos.

4. **La expresión no debe quedar con la pila vacía antes de tiempo.**

Si durante el recorrido intentas sacar un elemento y la pila está vacía, la expresión es incorrecta.

5. **Identificar correctamente operadores y operandos.**

Debes tener claro qué símbolos considerarás operadores (+, -, *, /, ^, etc.).

Todo lo demás podría tratarse como operando (variables o números), según los requisitos de tu proyecto.

6. **Operandos de varios caracteres.**

Si la expresión llega separada por espacios:

25 7 +

entonces 25 debe tratarse como un solo operando.

Si recorres la cadena carácter por carácter:

257+

entonces interpretarías 2, 5 y 7 como operandos distintos.

Esto depende del formato que acepte tu programa.

7. **Operadores unarios (si existen).**

El algoritmo que describiste supone operadores binarios (dos operandos).

Si en algún momento quisieras soportar operadores unarios, como:

-A

o una función como:

$\sin(x)$

la construcción cambia porque esos operadores solo requieren un hijo.

8. La expresión debe estar realmente en postfijo válido.

El algoritmo no convierte expresiones; únicamente construye el árbol suponiendo que la notación postfija ya es correcta.

En resumen, tu lógica es correcta. Los principales puntos críticos son:

- mantener el orden **izquierdo/derecho** al desapilar,
- verificar que existan suficientes operandos antes de procesar un operador,
- comprobar que al finalizar quede exactamente un nodo en la pila,
- y definir claramente cómo reconocer operadores y operandos (especialmente si habrá números de varios dígitos o variables).

Entre algunas decisiones tomadas resalto el hecho de agregar el apartado donde se obtiene el resultado de evaluación según lo que se ingrese, además de la opción de mostrar de forma más detallado el apartado de “enorden” mostrando la expresión con paréntesis que separan la expresión según formen raíces y las ramas de estas mismas.

Conclusión

El proceso de árbol de expresiones representa un proceso en el que toma una expresión posfija, lo que genera un orden donde se obtienen al menos dos datos que puedan resolver una operación y al menos un operador, por lo que, al existir esta relación de datos, se puede resolver estas operaciones a partir de un árbol que además, permite una representación más gráfica y con un orden jerárquico que permite una entrada similar a las pilas y, que de igual forma, permite presentar la expresión en diferentes órdenes.